

Three Threads

QA on the VeriContest dataset · ProofGen bottlenecks across the literature · NVIDIA CUDA on Rust

September 13, 2026

UIUC SRSE 2026 · Mahadi

QA on the VeriContest dataset

lynette rejects ground truth, on both suites

Last time: `lynette compare` rejects VeriContest's own reference proofs. Since then the same sweep has been run against a second benchmark, and the VeriContest corpus has been pinned and vendored so the number can be re-derived.

suite	pair compared	n	rejected	build · corpus
Verus-Bench	unverified.rs VS verified.rs	149	84 (56.4%)	2f9aff36 · compare -t
VeriContest	code_spec.rs VS verified.rs	946	298 (31.5%)	fd9edd2f0 · compare -t
			371 (39.2%)	-t -spec, benchmark@0cc50f5

These are the benchmarks' own answers

Every problem ships a reference proof that is, by construction, a correct answer to the task it poses. Nothing held to `lynette` could have produced the rejected ones.

A further **7** VeriContest problems `lynette` cannot parse at all.

They are counted separately: `compare` exits 1 for a parse failure and for a difference alike, so calling them rejections would report a verdict `lynette` never reached. Verus accepts all seven.

The pipeline runs `-t` alone, which never sets `spec`, so every `spec fn` is erased from both sides and no `spec-function` body is ever compared. Turning `-spec` on adds **73** rejections and loses none.

A rejection here is counted, not interpreted: which of the 371 are the `lynette`'s fault is a separate exercise, and is not what this section claims.

cf1200C: the whole objection is one import

A VeriContest reference proof, the benchmark's own answer, rejected by lynette. Deghosted, this is its *entire* difference from `code_spec.rs`:

verified.rs VS code_spec.rs, after erasure

```
use vstd::prelude::*;  
+ use vstd::arithmetic::div_mod::*;
```

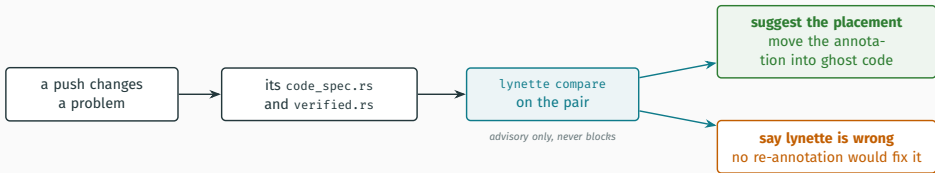
It exists solely to name three lemmas, all called *only* from `proof fn` bodies. Everything else erases cleanly and matches. lynette exits 1; Verus never runs.

Why. Verus separates **ghost** code, everything that exists only for the proof, from the program itself. An import written at the top of the file is not ghost code: it is ordinary module-level Rust, so lynette compares it like any other program text. The lemmas it names *are* ghost, and `erase`; the line that imports them does not.

The recommended form. Write the `use` *inside* the `proof fn` or `proof {}` that needs it, or fully qualify the call sites. lynette accepts; Verus still verifies.

Proposal: an advisory check on the dataset itself

Nothing here asks the authors to change how ProofGen is scored. It asks that a problem not enter the benchmark carrying an annotation lynette will object to: checked when the problem is written, not when a model is graded.



Why it belongs upstream

The benchmark's own consistency checker **strips use items before comparing**, so an import difference is invisible at authoring time and surfaces only later, as a model's `unsafe_code_or_spec_change`. The check is proposed as a PR to VeriContest.

Why it has to say when lynette is wrong

Not every rejection has a fix. Where the erasure itself manufactured the difference, there is no placement that satisfies lynette, and an advisory that only ever suggests re-annotation would send an author looking for one.

ProofGen bottlenecks across the literature

What every one of these papers is doing

One task, shared across the whole literature: **spec** + **code** → **proof**. The model is handed a program together with its specification and no proof, and must supply the annotations that make Verus accept it.



adds invariants, asserts and lemmas; the program and the contract may not change

What a benchmark is

A fixed set of such tasks, each shipping a reference proof written by a human. The reported number is the fraction of the set that Verus accepts.

Why the tasks are the variable

The mechanism differs from paper to paper: prompting, retrieval, tree search, counterexamples, agents. The task never does. So a result is a claim about a system *and* about the set of tasks it was run on.

Every paper, every suite it reports on

Each number is that paper's own, under its own model and Verus version; every system here generates and then repairs, so these are end-to-end task success rates. **Bold** = built from real systems code.

paper	benchmark	n	solved	kind
Yao et al. '23	Diffy-derived vector programs	20	38/55 seg	textbook, human in loop
AlphaVerus '24	MBPP	–	65.7%	textbook
	HumanEval	–	32.9%	textbook
AutoVerus '25	Verus-Bench	150	91.3%	textbook + Diffy
SAFE '25	VerusBench	139	52.5%	textbook
	CodeNet-Test	1,435	48.5%	synthesised
RagVerus '25	VerusBench	139	60.4%	textbook
	RepoVBench	383	19.6%	VeriSMo, confidential VM
VeruSAGE '25	VeruSAGE-Bench	849	81%	8 verified systems
ExVerus '26	VerusBench	146	88.4%	textbook + Diffy
	Dafny2Verus	67	95.5%	textbook
	HumanEval-Verus	68	41.2%	textbook
	LCBench	28	28.6%	LeetCode, ~200 LoC
	ObfsBench	266	> 73%	obfuscated VerusBench
	InvariantInjectBench	187	–	<i>repair only</i> : seeded bad invariant
KVerus '26	Verus-Bench	150	88%	textbook + Diffy
	MBPP	78	83%	textbook
	Human-Eval	85	64%	textbook
	MathSpec-Bench	–	77.9%	Lean 4 maths, repo
	Memory Allocator	–	56.2%	allocator, repo
	CortenMM	–	31.3%	Asterinas kernel MM
VeriContest '26	VeriContest	946	13.95%	LeetCode + Codeforces

A mechanism only shows its strength on a hard enough dataset

One paper, one model, one harness, four datasets, so the only thing varying is the difficulty of the tasks. The better mechanism wins by more and more as the dataset gets harder, and by nothing at all once the dataset is saturated.

ExVerus vs AutoVerus, Sonnet-4.5 success rate, %

dataset	n	AutoVerus	ExVerus
Dafny2Verus	67	95.5	95.5
VerusBench	146	75.3	88.4
HumanEval-Verus	68	27.9	41.2
LCBench	28	14.3	28.6

On a saturated dataset the mechanism is invisible.

Dafny2Verus is already at 95.5 for both, so no technique can show anything there.

The harder the tasks, the more the mechanism is worth.

On LCBench ExVerus **doubles** AutoVerus. Progress is only measurable where there is room to measure it.

A dissection of the kinds of problems current datasets provide

kind of problem	suites	tasks	what one task looks like
competitive programming	VeriContest · CodeNet-Test · LCBench	946 · 1,435 · 28	one function, a loop over an array, an interview or contest algorithm
entry-level / textbook	MBPP · HumanEval-Verus · CloverBench · Dafny2Verus	78 · 68 · 11 · 67	string and list operations, sorting, searching, array transforms
invariant-stress arrays	Diffy	38	accumulator loops with deliberately awkward inductive relationships
perturbations of the above	ObfsBench · InvariantInjectBench	266 · 187	the same problems, obfuscated, or seeded with one bad invariant
real systems code	VeruSAGE-Bench · RepoVBench · KVerus repo-level	849 · 383 · 359	OS kernels, allocators, page tables, a confidential-VM security module, a binary parser

The two are not the same exercise

	Verus-Bench	VeruSAGE-Bench
LoC per task	32	~947
lemmas per task	0.07	2.4

Only **three** suites are built from systems code, and only one of them is substantial. A mechanism is only visible on hard tasks, and the field has very few. **What the literature lacks is relevant datasets, more than it lacks proof techniques.**

So Mahadi, can you improve ProofGen or not?

What I set out to do

Agentic-loop mechanisms, aimed at VeriContest, to beat **13.95%**.

What reading the papers did to that

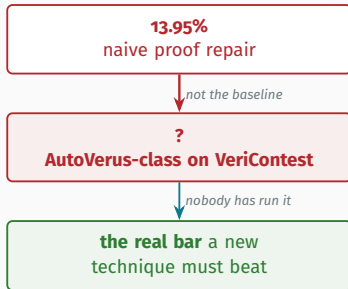
The draft ideas are already in the literature, and implemented better than I had them: ExVerus's counterexample-guided repair and RagVerus's retrieval both go further than what I had drawn up.

Can I still do better on current datasets?

About 70% confident yes, and *how* is a different talk from this one.

But then what would the result mean?

Suppose I add technique, beat a current number by a margin, and run the same thing on VeriContest. Against what do I claim the win?



13.95% is not the number to beat: it is what a naive proof-repair loop scores. The honest comparison is to run the established mechanisms on VeriContest first, find out what *they* score, and beat that. **And if ProofGen is hard to improve substantially, what then?** That is the next section.

NVIDIA CUDA on Rust

Contribute a benchmark dataset for LLM-written CUDA kernels in Rust

Section 2's conclusion, applied

Rust exists to write **low-level systems** safely. The ProofGen literature barely evaluates on that kind of code: three suites, one of them substantial.

And GPU kernels are certainly not among them. They *could not* be: NVIDIA brought CUDA to Rust on **8 September 2026**, five days ago.

So what I am doing

A **feasibility study** of whether formal verification is needed here at all, started from knowing nothing about GPU kernels.

Academia runs on *publish or perish*. As a Fall '27 applicant with no publication, mine for the next 60 days is **submit or perish**, which is why a new benchmark dataset is the bet I think is worth making.

Why a dataset rather than a technique

Section 2: a technique has to beat an established number on an established suite. A dataset that does not exist yet has no incumbent to displace.

What this section asks

1. Is there scope, or need, for **functional correctness** of CUDA kernels at all?
2. Does that change when the kernel is written in **Rust**?
3. Can **Verus** handle it?
4. If it can, where would adding `requires` / `ensures` actually be interesting?

ProofWright: agentic formal verification of CUDA

arXiv 2511.12294, November 2025. Formal verification of **LLM-generated CUDA kernels**, because runtime testing has limited coverage and is “*vulnerable to reward hacking*.”

What it does

VerCors Agent synthesises deductive annotations from a knowledge base, an annotation guide and an error-fix database, giving SMT-backed **memory safety and data-race freedom**.

Rocq Agent extracts the computational specification from the PyTorch source and proves **semantic equivalence** against the generated CUDA, on MLRocq, a verified library of tensor operators.

On KernelBench L1 100 problems, Claude-4-Sonnet

verified: memory & thread safety	74%	~3 min/kernel
SMT solver instability	17%	
agent failure	9%	

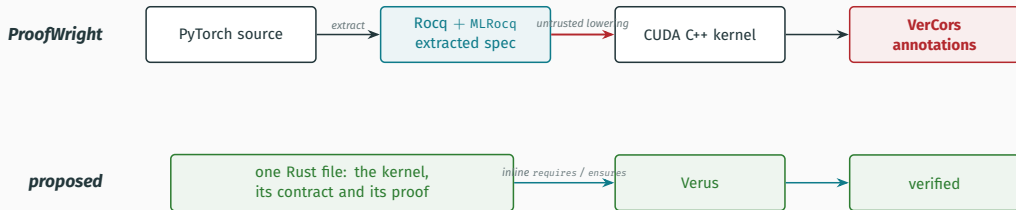
What its authors say about Verus

They compare themselves directly to AutoVerus, and make three claims.

1. Verus, like VerCors, “suffers from low amounts of training data.”
2. Verus programs get Rust’s type-system guarantees, so they **need not prove memory safety or data-race freedom** for safe Rust, less burden on the LLM to reason about concurrency or memory safety.
3. AutoVerus *assumes* the specification is already given in Verus; ProofWright extracts the functional specification from a high-level implementation.

CUDA on Rust collapses that pipeline

ProofWright's architecture is *forced* by CUDA being C++: the specification lives in PyTorch, the proof lives in Rocq, the kernel lives in CUDA: three languages, two verifiers, and an *untrusted lowering stage* between them, which the paper itself flags.



What collapsing it buys. No specification to extract from a high-level framework. No second, interactive theorem prover. No lowering gap between the thing proved and the thing that runs.

And by their own argument, the easy half is already done. Safe Rust gives memory safety and data-race freedom for free, so the work moves to exactly where ProofWright got least far: **functional correctness**.

Have I checked a real codebase?

Verus is already in industry

Microsoft, Amazon. **VeriSMo** and **Anvil**: two of three OSDI'24 best papers. **CortenMM**: concurrent memory management.

The intuition

VeriSMo's 869 requires sit at **trust boundaries**, not everywhere. `external_body` means: *body unverified, contract enforced*.

Verification is not proving everything. It is

bounding what you trust.

The same placement, on a cuTile MoE kernel

where	contract	what it buys
descriptor loaders	requires	five unchecked <code>assume</code> axioms become caller obligations
kernel entry	requires	restores the shape agreement the code computes, then discards
scheduler loop	invariant	div-by-zero on an empty expert; the tile partition
<code>mma</code> , <code>pointer</code> & <code>tensor-view</code> primitives	<code>external_body</code> + <code>contract</code>	bounds the trusted base; float semantics stay <i>below</i> it

GPU kernels already mark their trust boundaries, with `unsafe` and with `assume`.

The contracts are already written, as assume, not requires

[cutile-kernels/src/experimental/moe.rs](https://github.com/nvidia/cutile-kernels/src/experimental/moe.rs), at 2eed75e. NVIDIA's own comment above these: *"compile-time hints the optimizer treats as facts. UB if the property doesn't hold at runtime, hence unsafe."*

today: asserted inward, unchecked

```
let rows: i32 = unsafe {
    assume_div_by::(<_, 16>{
        assume_bounds_lower::(<_, 0>{
            tile_to_scalar(rows_tile)))
    };
};
```

Three of these in one function (rows, cols, stride0), plus one on the pointer. Each states a fact and **declines to check it**.

Disclaimer: the right-hand contract is my own draft, worked out with LLM help: intuition from a few days of studying CUDA, not a verified result.

the same facts, demanded outward

```
unsafe fn load_f16_desc_2d(..., group_id: i32)
-> (res: (Tensor<f16,{[-1,-1]}>, i32, i32))
requires
    0 <= group_id < metas@.len(),
    metas@[group_id as int][0] % 16 == 0, // rows
    metas@[group_id as int][1] % 16 == 0, // cols
    metas@[group_id as int][2] % 8 == 0, // stride
    ptrs@[group_id as int] % 16 == 0, // align
ensures
    res.1 >= 0 && res.2 >= 0,
```

Why the dataset looks like the better bet

- On saturated suites, no technique shows anything.
- Three benchmarks are built from real systems code. None is a GPU kernel, and none could be.
- The contracts are already in that code, as unchecked `assumeS`.

And the M2 spec testing the group works on

It is **ProofGen on a subset**: writing proofs for the validity of pre- and post-conditions. The reuse-and-apply patterns I have seen are close to what **AutoVerus** and **RagVerus** already do.

So the first thing to do: take **50–100 spec-testing problems**, apply the state-of-the-art ProofGen techniques, and find out whether they already work well.

I am open to any sub-track, spec testing included, where the chance of a publishable result inside the window is high.

